

# Basics of Neural Network Programming

Syed Hamed Raza

University of Management and Technology (UMT), Lahore

Course: Deep Learning and Neural Network (CS4152)

November 6, 2025

# Why Learn Neural Network Programming Basics?

- Neural networks require efficient computation techniques.
- Avoid explicit `for` loops over training examples.
- Use **vectorization** to process entire training sets.
- Understand the two key steps: **Forward Pass** and **Backward Pass**.

# Forward and Backward Propagation

| Step                 | Purpose                                       |
|----------------------|-----------------------------------------------|
| Forward Propagation  | Compute predictions ( $\hat{y}$ ) from inputs |
| Backward Propagation | Compute gradients to update weights           |

- These steps form the **core learning cycle**.
- Apply from logistic regression to deep neural networks.

# Using Logistic Regression as an Example

- Logistic regression is a **binary classifier**.
- Simplifies understanding of neural network computations.
- Outputs:
  - $y = 1$  : Cat
  - $y = 0$  : Not Cat

# Representing an Image as Input

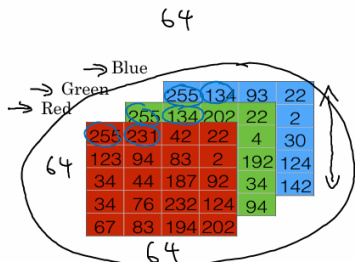
- Images are stored as three matrices (R, G, B channels).
- Example:  $64 \times 64$  pixels  $\times$  3 channels.
- Each pixel intensity value ranges from 0 to 255.
- Flatten (unroll) into a single long feature vector  $x$ .

## Binary Classification



1 (cat) vs 0 (non cat)

$y$



$$X = \begin{bmatrix} 255 \\ 231 \\ \vdots \\ 255 \\ 134 \\ \vdots \end{bmatrix}$$

$$64 \times 64 \times 3 = 12288$$

$$n = n_x = 12288$$

$$X \longrightarrow y$$

# Constructing the Feature Vector

$$x = [r_1, r_2, \dots, g_1, g_2, \dots, b_1, b_2, \dots]^T$$

- Total dimension:  $n_x = 64 \times 64 \times 3 = 12,288$ .
- $x \in \mathbb{R}^{n_x}$  is the input to the model.

# Binary Classification Goal

$$\hat{y} = \begin{cases} 1 & \text{if cat} \\ 0 & \text{otherwise} \end{cases}$$

- $y \in \{0, 1\}$  is the label.
- Goal: Learn a classifier  $f(x)$  that predicts  $y$  accurately.



# Notation Setup

- Single training example:  $(x^{(i)}, y^{(i)})$
- Training set:  $\{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}$
- $m = m_{train}$ : Number of training examples.
- $m_{test}$ : Number of test examples.

# Organizing the Data: Input Matrix $X$

$$X = [x^{(1)} \ x^{(2)} \ \dots \ x^{(m)}]$$

- $X \in \mathbb{R}^{n_x \times m}$ .
- Each column represents one training example.
- This convention simplifies vectorized NN implementation.

# Organizing Labels: Output Y

$$Y = [y^{(1)} \ y^{(2)} \ \dots \ y^{(m)}]$$

- $Y \in \mathbb{R}^{1 \times m}$ .
- Each column corresponds to label of its input example.
- Keep X and Y aligned column-wise for easy computation.

# Notation

$$(x, y) \quad x \in \mathbb{R}^{n_x}, y \in \{0, 1\}$$

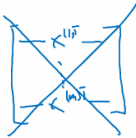
$$m \text{ training examples} : \{(\underline{x}^{(1)}, \underline{y}^{(1)}), (\underline{x}^{(2)}, \underline{y}^{(2)}), \dots, (\underline{x}^{(m)}, \underline{y}^{(m)})\}$$

$$M = M_{\text{train}}$$

$$M_{\text{test}} = \# \text{test examples.}$$

$$X = \begin{bmatrix} | & | & & | \\ x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ | & | & & | \end{bmatrix} \begin{matrix} \uparrow \\ n_x \\ \downarrow \end{matrix}$$

$X \in \mathbb{R}^{n_x \times m}$        $X.\text{shape} = (n_x, m)$



$$Y = [y^{(1)} \ y^{(2)} \ \dots \ y^{(m)}]$$

$$Y \in \mathbb{R}^{1 \times m}$$

$$Y.\text{shape} = (1, m)$$

Figure: Enter Caption

# Python Shapes Recap

| Quantity | Shape      | Meaning                            |
|----------|------------|------------------------------------|
| $X$      | $(n_x, m)$ | Inputs: features $\times$ examples |
| $Y$      | $(1, m)$   | Labels                             |

Example: `X.shape`  $\rightarrow$   $(12288, m)$

# Summary and Next Steps

- Efficient implementation = Vectorization + Matrix Notation.
- Data stored column-wise for forward/backward propagation.
- Next: Formulate Logistic Regression Model using these conventions.